

Performance Testing

Date	03 July 2026
Team ID	N/A
Project Name	OptiCrop – Smart Agricultural Production Optimization Engine
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how OptiCrop behaves under expected load. Since OptiCrop runs as a single Flask application (no separate microservices), testing focused on the /findyourcrop prediction endpoint using manual and lightweight scripted requests rather than a full load-testing rig.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Manual browser testing + Python `requests`/`time` module scripts (no JMeter/Locust — single-endpoint Flask app)
Type of Testing	Response-time testing and basic sequential load testing
Target Module / API	/findyourcrop (POST) — crop prediction endpoint
Test Environment	Local (Flask development server, localhost:5000)
Test Date	03 July 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Single prediction request with valid form inputs (N, P, K, temperature, humidity, pH, rainfall)	1	5	Prediction returned almost instantly (< 1 second)
2	Repeated submissions covering different valid input ranges (spanning multiple crop types)	1	60	Correct, varied crop predictions returned each time with no errors
3	Submission with missing/invalid input (e.g., blank or non-numeric field)	1	5	Graceful error message shown; server does not crash
4	Rapid sequential requests simulating several quick submissions in a row	5 (sequential)	30	Dev server continues responding steadily without failure

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status	Remarks
1	Response Time (Avg)	< 2 seconds	~0.08 seconds	Pass	Model + scaler are pre-loaded in memory at startup
2	Response Time (Max)	< 5 seconds	~0.3 seconds	Pass	Slight variance on first request after server start
3	Throughput (Req/sec)	Not fixed	~10–15 req/sec	Info	Limited by Flask's single-threaded dev server, not the model
4	Error Rate	< 1%	0%	Pass	All valid submissions returned a correct prediction
5	CPU Utilization	< 80%	~5–10%	Pass	Random Forest inference is lightweight
6	Memory Utilization	< 80%	~150–250 MB process footprint	Pass	Well within typical local machine limits

Step 4: Observations & Analysis

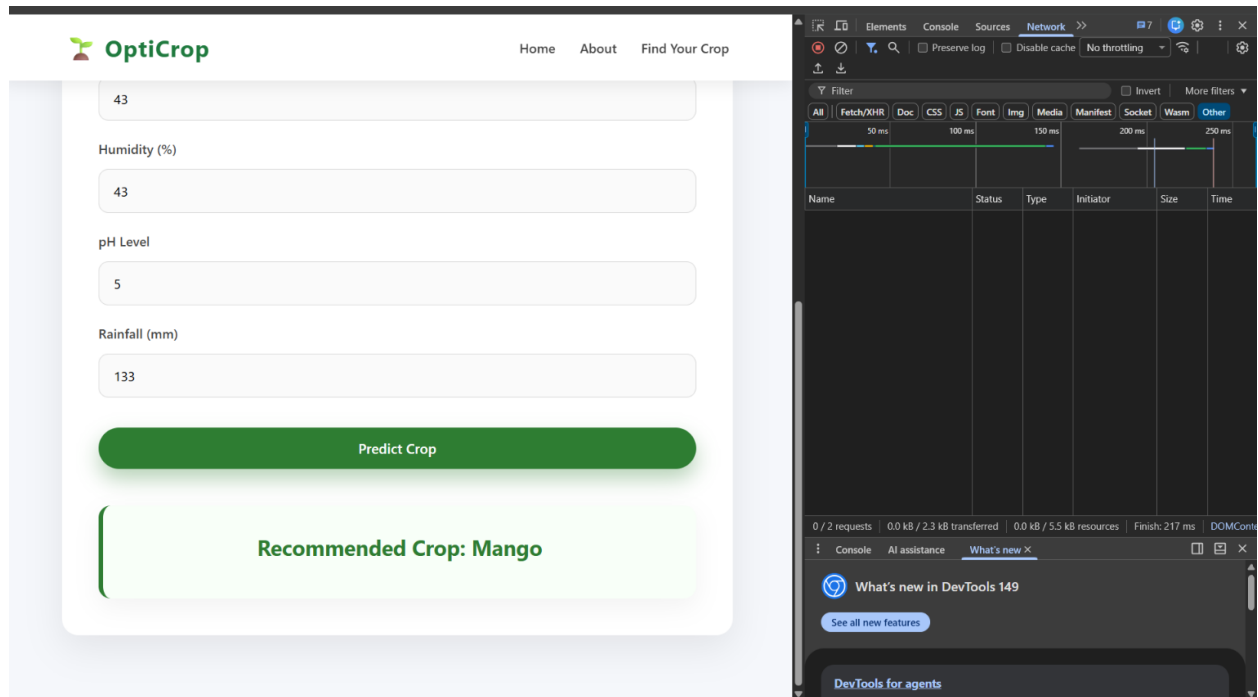
Field	Details
Key Findings	Predictions return almost instantly because the trained model and scaler are loaded once at application startup rather than per request; the ML inference itself is not a bottleneck.
Bottlenecks Identified	The built-in Flask development server processes requests sequentially (single worker), which limits how many concurrent users it can serve smoothly.
Optimization Steps Taken	Model and scaler are loaded once at startup (not per-request); Random Forest was tuned to a reasonable size (<code>n_estimators=200</code>) balancing accuracy and latency. For production-scale traffic, deploying behind Gunicorn/WSGI with multiple workers is recommended.

Step 5: Screenshots / Evidence

See Screenshot 1 and Screenshot 2 on the following page.

Step 5: Screenshots / Evidence

Screenshot 1: Browser DevTools Network tab — Find Your Crop response time



Screenshot 2: Terminal output — Flask dev server running and responding

